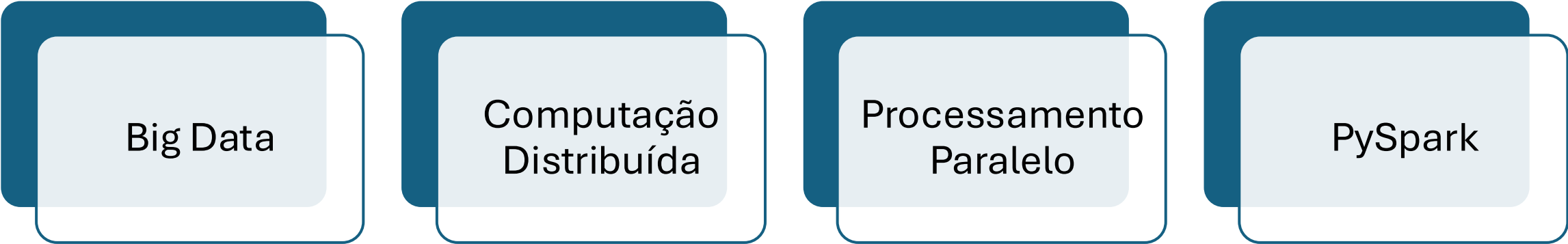


Princípios de Desenvolvimento de Spark com Python

Prof.: Felipe Alves

Conceitos, Arquitetura e Aplicações com PySpark



Big Data

The diagram consists of four light blue rounded rectangular boxes arranged horizontally. Each box is partially overlapped by a dark blue rounded rectangular shape behind it, creating a layered effect. The boxes contain the text 'Big Data', 'Computação Distribuída', 'Processamento Paralelo', and 'PySpark' from left to right.

Computação
Distribuída

Processamento
Paralelo

PySpark

O que é Big Data?

Conceito de Big Data

- Grande volume de dados
- Alta velocidade de geração
- Diversidade de formatos
- Necessidade de processamento eficiente

Desafios

- Armazenamento
- Processamento
- Escalabilidade
- Tempo de resposta

Surgimento do Apache Spark

Por que o Spark foi criado?

- Limitações do Hadoop tradicional
- Necessidade de maior velocidade
- Melhor uso da memória RAM
- Processamento em tempo real

Objetivos

- Alto desempenho
- Tolerância a falhas
- Processamento distribuído

O que é Apache Spark?

Framework de Computação Distribuída

- Código aberto
- Desenvolvido pela Apache Foundation
- Processa grandes volumes de dados
- Suporte a múltiplas linguagens

Linguagens suportadas

- Python
- Scala
- Java
- R

Principais Vantagens do Spark

Benefícios do Spark

- Processamento em memória
- Alta velocidade
- APIs simples
- Bibliotecas integradas

Aplicações

- Machine Learning
- Streaming
- SQL
- Processamento de grafos

Hadoop vs Spark

Hadoop	Spark
Processamento em disco	Processamento em memória
Mais lento	Mais rápido
Batch tradicional	Batch + Streaming
Menor consumo RAM	Maior consumo RAM

- **Conclusão**
- Spark é mais eficiente para análises modernas.

Principais Cenários de Uso

- **Onde o Spark é utilizado?**
 - ETL
 - Streaming
 - Ciência de Dados
 - Machine Learning
 - Processamento de logs
 - IoT
 - Redes sociais



Cenários Inadequados

Quando o Spark pode **não** ser ideal?

- Pouca memória RAM
- Cluster limitado
- Infraestrutura simples
- Pequeno volume de dados

Observação

- Spark exige boa configuração de cluster.

Arquitetura do Spark

- **Como o Spark funciona internamente?**

A arquitetura do Apache Spark foi criada para permitir que grandes volumes de dados sejam processados em vários computadores ao mesmo tempo.



Componentes principais



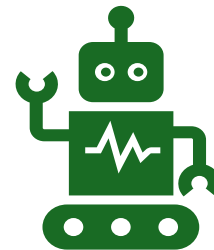
Driver Program

É o “cérebro” da aplicação.

Recebe o código desenvolvido pelo programador.

Organiza as tarefas que serão executadas.

Coordena todo o processamento.



Cluster Manager

Gerencia os recursos do cluster.

Decide quais máquinas serão utilizadas.

Distribui memória e processamento.

Componentes principais



Worker Nodes

São os computadores que executam as tarefas.

Cada Worker pode executar várias operações simultaneamente.



Executors

Processos que executam efetivamente os cálculos.

Trabalham dentro dos Workers.
Retornam os resultados ao Driver.



Ideia principal

O Spark divide o processamento entre várias máquinas para ganhar velocidade e escalabilidade.

O que é PySpark?

Interface Python do Spark

- API Python para Spark
- Permite usar Spark com Python
- Integração com ciência de dados

Vantagens

- Simplicidade
- Bibliotecas Python
- Alto desempenho

Fluxo de Execução no Spark

O que acontece quando
executamos um programa
Spark?



Etapa 1 — Aplicação é iniciada

O usuário executa um
código PySpark.

Exemplo:

- `spark.read.csv("dados.csv")`

Fluxo de Execução no Spark

Etapa 2 — Driver cria o DAG

- Gráfico Acíclico Dirigido (do inglês Directed Acyclic Graph – DAG).
- O DAG é um mapa criado pelo Spark para organizar o processamento.

O Driver analisa o código e cria um plano de execução chamado DAG.

- O DAG organiza:
 - quais operações serão feitas;
 - a ordem correta;
 - como distribuir o processamento.

Função do DAG

O DAG define:

- quais operações serão executadas;
- em qual ordem;
- como otimizar o processamento.

Benefícios

- evita processamento desnecessário;
- melhora desempenho;
- reduz uso de memória;
- aumenta paralelismo.

Fluxo de Execução no Spark

Etapa 3 — Cluster Manager distribui recursos

- O Cluster Manager:
- seleciona os Workers;
- libera memória;
- ativa os Executors.

Etapa 4 — Workers executam tarefas

- Os dados são divididos em partes menores.
- Cada Worker:
- recebe parte dos dados;
- executa cálculos em paralelo;
- processa sua partição.

Fluxo de Execução no Spark

Etapa 5 — Resultados retornam

- Os resultados processados retornam ao Driver Program.

Resultado final

- O usuário recebe:
- relatórios;
- tabelas;
- estatísticas;
- arquivos processados.

Vantagem

- Diversos computadores trabalham simultaneamente.

Entendendo o RDD

O que é RDD?

- **(Resilient Distributed Dataset)**
- Conjunto de dados distribuídos resilientes

Ou seja:

- Resiliente
- Distribuído
- Conjunto de Dados

**Por que o
RDD é
importante?**

O RDD é a principal estrutura de dados do Spark.



Ele permite:

dividir dados
em várias
máquinas;

processar
dados em
paralelo;

recuperar
dados em caso
de falha.

Principais características do RDD

Distribuído

- Os dados são divididos entre vários computadores.

Resiliente

- Se uma máquina falhar, o Spark reconstrói os dados automaticamente.

Imutável

- O RDD não pode ser alterado diretamente. Toda modificação gera um novo RDD.

Paralelo

- Muitas tarefas são executadas simultaneamente.

Benefício:

- Permite processar grandes volumes de dados com alta eficiência.

Ecossistema do Spark

O Spark possui vários módulos especializados

O Spark não é apenas uma ferramenta única.

Ele possui um ecossistema completo de bibliotecas.

Ecossistema do Spark

Spark Core

- **Núcleo principal do Spark**

Responsável por:

- gerenciamento de memória;
- execução das tarefas;
- tolerância a falhas;
- processamento distribuído.
- É a base de todos os outros módulos.

Ecosistema do Spark

Spark SQL

- **Processamento de dados estruturados**

Permite:

- consultas SQL;
- uso de DataFrames;
- integração com bancos de dados.

Exemplo:

- `spark.sql("SELECT * FROM tabela")`

Ecosistema do Spark

Spark Streaming

- **Processamento em tempo real**

Usado para:

- logs;
- sensores IoT;
- monitoramento;
- redes sociais;
- fraudes financeiras.

Os dados chegam continuamente.

Ecossistema do Spark

MLlib

Machine Learning no Spark

Biblioteca para:

- classificação;
- regressão;
- agrupamento;
- recomendação;
- análise estatística.

Ecossistema do Spark

GraphX

- **Processamento de grafos**

Utilizado para:

- redes sociais;
- conexões;
- análise de relacionamentos;
- grafos distribuídos.

Grande vantagem do ecossistema

- Todos os módulos trabalham integrados no mesmo ambiente Spark.

Conclusão

Principais Aprendizados

- Spark é referência em Big Data
- PySpark facilita desenvolvimento
- Processamento distribuído aumenta desempenho
- RDD e DAG são conceitos centrais

Aplicações modernas usam Spark em:

- IA , Streaming, ETL e Ciência de Dados

Encerramento

- “O Spark transformou a forma de processar grandes volumes de dados.”